You can change the default value with the setter method default=, which takes any arbitrary value on the right side and makes it the new default for missing keys. Similarly, the default_proc= method takes a proc and makes it the new default. As with Hash.new the proc takes two arguments, the receiving hash, and the key being sought. There are getters, default and default_proc that return the current value of the defaults.

If you want to retrieve values from multiple keys at the same time, the values_at method takes multiple keys and returns an array of the values at those keys, in order, returning nil or the default value for keys that don't exist. The fetch_values method does the same thing, except it takes a block that is invoked for missing keys, and if there's no block, it raises a KeyError:

```
hash = {a: 1, b: 2, c: 3, d: 4}
hash.values_at(:b, :g, :d)                        # => [2, nil, 4]
hash.fetch_values(:b, :g, :d) { |key| key.to_s } # => [2, "g", 4]
```

Hashes also respond to dig by looking up the first argument in the hash and passing the remainder of the arguments to that result.

If for some reason you want the key value pair, the assoc method takes a key and returns the matching key/value pair as a two-element array.

If you have the value and want the matching key, the key method returns the first key in order that has that value. The rassoc method gives you the entire key/value pair as an array (we'll cover hash ordering in Iterating Hashes, on page 549 and some boolean queries for hashes in Querying a Hash, on page 549):

```
hash = {a: 1, b: 2, c: 3, d: 4}
hash.assoc(:b) # => [:b, 2]
hash.key(2)    # => :b
hash.rassoc(2) # => [:b, 2]
```

Setting Values

You set a value in a hash using the []= method, which is aliased as the method store. When you call []=, if the key in the brackets exists in the hash, the value associated with that key is updated. If the key does not exist in the hash, it is added with the value, and it is appended to the end of the hash for ordering purposes.

To completely remove a key from the hash, the delete method takes the key. If the key is in the hash, it removes the key and returns the associated value, otherwise it returns nil. An optional block is called with the key if the key is not in the hash, that value is returned:

```
hash = {a: 1, b: 2, c: 3, d: 4}
hash[:b] = 700
hash[:e] = 5
hash.delete(:c) # => 3
hash            # => {:a=>1, :b=>700, :d=>4, :e=>5}
```

A related method, delete_if takes a block with a key and a value and removes every element from the hash for which the block returns true:

```
hash = {a: 1, b: 2, c: 3, d: 4}
hash.delete_if { |key, value| value.even? } # => {:a=>1, :c=>3}
hash                                        # => {:a=>1, :c=>3}
```